FPRaker: A Processing Element For Accelerating Neural Network Training

Omar Mohamed Awad*
University of Toronto
omar@cerebras.net

Mostafa Mahmoud
University of Toronto
mostafa.mahmoud@mail.utoronto.ca

Isak Edo†
University of Toronto
isak.edo@mail.utoronto.ca

Ali Hadi Zadeh
University of Toronto
a.hadizadeh@mail.utoronto.ca

Ciaran Bannon
University of Toronto
ciaran.bannon@mail.utoronto.ca

Anand Jayarajan
University of Toronto
anandj@cs.toronto.edu

Gennady Pekhimenko
University of Toronto, Vector Institute
pekhimenko@cs.toronto.edu

Andreas Moshovos
University of Toronto, Vector Institute
moshovos@ece.utoronto.ca

ABSTRACT

We present *FPRaker*, a processing element for composing training accelerators. *FPRaker* processes several floating-point multiply-accumulation operations concurrently and accumulates their result into a higher precision accumulator. *FPRaker* boosts performance and energy efficiency during training by taking advantage of the values that naturally appear during training. It processes the significand of the operands of each multiply-accumulate as a series of signed powers of two. The conversion to this form is done on-the-fly. This exposes ineffectual work that can be skipped: values when encoded have few terms and some of them can be discarded as they would fall outside the range of the accumulator given the limited precision of floating-point. *FPRaker* also takes advantage of spatial correlation in values across channels and uses delta-encoding off-chip to reduce memory footprint and bandwidth. We demonstrate that *FPRaker* can be used to compose an accelerator for training and that it can improve performance and energy efficiency compared to using optimized bit-parallel floating-point units under iso-compute area constraints. We also demonstrate that *FPRaker* delivers additional benefits when training incorporates pruning and quantization. Finally, we show that *FPRaker* naturally amplifies performance with training methods that use a different precision per layer.

ACM Reference Format:
Omar Mohamed Awad, Mostafa Mahmoud, Isak Edo, Ali Hadi Zadeh, Ciaran Bannon, Anand Jayarajan, Gennady Pekhimenko, and Andreas Moshovos. 2021. FPRaker: A Processing Element For Accelerating Neural Network Training. In *MICRO'21: 54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '21), October 18–22, 2021, Virtual Event, Greece.* ACM, New York, NY, USA, 13 pages. https://doi.org/10.1145/3466752.3480106

*Also with Cerebras Systems. Work completed while at the University of Toronto.
†Also with Arm. Work completed while at the University of Toronto.

MICRO '21, October 18–22, 2021, Virtual Event, Greece

ACM ISBN 978-1-4503-8557-2/21/10…$15.00
https://doi.org/10.1145/3466752.3480106

1 MOTIVATION

The pervasive applications of deep learning and the end of Dennard scaling have been driving efforts for accelerating deep learning inference and training. These efforts span the full system stack, from algorithms, to middleware and hardware architectures. Here we target training, a task that includes inference as a subtask. Training is a compute- and memory-intensive task often requiring weeks of compute time even with state-of-the-art training methods and hardware [52]. The cost of training is staggering. For example, training XLNet requires 512 TPU v3 chips for 2.5 days leading to a total training cost above $61K [57]. Further, training Grover-Mega requires 128 TPU v3 chips for 2 weeks leading to a total training cost of $25K [58]. Often multiple trials are required for hyperparameter tuning further amplifying costs.

During training a set of annotated inputs, that is inputs for which the desired output is known is processed by repeatedly performing a forward and backward pass. The *forward* pass performs inference whose output is initially inaccurate. However, given that the desired outputs are known, the training algorithm can calculate a *loss*, a metric of how far the outputs are from the desired ones. During the *backward* pass, this loss is used to adjust the network's parameters and to have it slowly converge to its best possible accuracy.

Numerous methods were developed to accelerate training, and fortunately often they can be used in combination. They use combination of software level techniques to reduce training time. On the other hand, the need to boost energy efficiency during inference has led to techniques that increase computation and memory needs during training. This includes works that perform network pruning and quantization during training and network architecture search.

Despite the successes already reported via the aforementioned methods, and while we expect that these methods will be refined, the need to further accelerate training both at the data center and at the edge remains unabated. Operating and maintenance costs, latency, throughput, and node count are major considerations for data centers. At the edge energy and latency are major considerations where training may be primarily used to refine or augment already trained models [6, 27, 56]. Regardless of the target application, improving node performance would be of value. Accordingly,

in this work we target methods that could *complement* existing training acceleration methods.

The bulk of the computations and data transfers during training is for performing multiply-accumulate operations (MAC) during the forward and backward passes. Compression methods can greatly reduce the cost of data transfers. Here we target the processing elements for these operations and propose designs that exploit *ineffectual* work that occurs naturally during training and whose frequency is amplified by quantization, pruning, and selective backpropagation.

A processing element that can eliminate ineffectual work during training may attempt to build upon the numerous proposals that exploit ineffectual work during *inference*. We highlight some those approaches that are most relevant to this work. Any MAC operation where any of the two input operands, be it the activation or the weight is ineffectual. The first class of accelerators rely on that zeros occur naturally in the activations of many models especially when they use ReLU [3, 9, 22, 29]. There are several accelerators that target pruned models e.g., [2, 8, 19, 21, 22, 31, 36, 41, 44, 59, 60]. Another class of designs benefit from reduced value ranges whether these occur naturally or result from quantization. This includes bit-serial designs [15, 28, 33, 34, 49], and designs that support many different datatypes such as BitFusion [50]. Another class of designs targets *bit-sparsity* where by decomposing multiplication into a series of shift-and-add operations they expose ineffectual work at the bit-level [2, 13, 48].

While we can draw from the experience gained from the aforementioned design for inference, training presents us with different challenges. First, is the *datatype*. While models during inference work with fixed-point values of relatively limited range, the values training operates upon tend to be spread over a large range. Accordingly, training implementations use floating-point arithmetic with single-precision IEEE floating point arithmetic (FP32) being sufficient for virtually all models. Other datatypes that facilitate the use of more energy- and area-efficient multiply-accumulate units compared to FP32 have been successfully used in training many models. These include bfloat16, and 8b or smaller floating-point formats [11, 12, 20, 24, 30, 53, 54]. Moreover, since floating-point arithmetic is a lot more expensive than integer arithmetic, mixed datatype training methods use floating-point arithmetic only sparingly [11, 14, 37, 40]. Despite these proposals, FP32 remains today the standard fall-back format, especially for training on large and challenging datasets. As a result of its limited range and the lack of an exponent, the fixed-point representation used during inference gives rise to zero values (too small a value to be represented), zero bit prefixes (small value that can be represented), and bit sparsity (most values tend to be small and few are large) that the aforementioned inference accelerators rely upon. FP32 can represent much smaller values, its mantissa is normalized, and whether bit sparsity exists has not been demonstrated.

Second, is the *computation structure*. Inference operates on two tensors, the weights and the activations, performing per layer a matrix/matrix or matrix/vector multiplication or pairwise vector operations to produce the activations for the next layer in a feed-forward fashion. Training includes this computation as its *forward* pass which is followed by the *backward* pass that involves a third tensor, the gradients. Most importantly, the backward pass uses the activation and weight tensors in a different way than the forward pass, making it difficult to pack them efficiently in memory, more so to remove zeros as done by inference accelerators that target sparsity. Related to computation structure, third, is *value mutability* and *value content*. Whereas in inference the weights are static, they are not so during training. Furthermore, training initializes the network with random values which it then slowly adjusts. Accordingly, one cannot necessarily expect the values processed during training to exhibit similar behavior such as sparsity or bit-sparsity. More so, for gradients which are values that do not appear at all during inference.

Our first contribution is that we demonstrate that a large fraction of the work performed during training is ineffectual. To expose this ineffectual work we decompose each multiplication into a series of single bit multiply-accumulate operations. This reveals two sources of ineffectual work: First, more than 85% of the computations are ineffectual since one of the inputs is zero. Second, the combination of the high dynamic range (exponent) and the limited precision (mantissa) often yields values which are non-zero, yet too small to affect the accumulated result, even when using extended precision (e.g., trying to accumulate 2^{-64} into 2^{64}).

This observation led us to consider whether it is possible to use *bit-skipping* (bit-serial where we skip over zero bits) processing to exploit these two behaviors. Bit-skipping processing of multiply-accumulate operations has been proposed before for inference. Bit-Pragmatic is a data-parallel processing element that performs such bit-skipping of one operand side [2] whereas Laconic does so for both sides [48]. Since these methods target inference only they work with fixed-point values. Since we found that there is little bit-sparsity in the weights during training, we focused on Bit-Pragmatic's approach. Converting a fixed-point design to floating-point is a non-trivial task to start with. Regardless, converting Bit-Pragmatic into floating-point resulted in an area-expensive unit which performs poorly under iso-compute area constraints. Specifically, compared to an *optimized* Bfloat16 processing element (see Section 5) that performs 8 MAC operations, under iso-compute constraints, an optimized accelerator configuration using the Bfloat16 Bit-Pragmatic PEs is on average 1.72× slower and 1.96× less energy efficient. In the worst case, the Bfloat16 bit-pragmatic PE was 2.86× slower and 3.2× less energy efficient. The Bfloat16 Bit-Pragmatic PE is 2.5× smaller than the bit-parallel PE, and while we can use more such PEs for the same area, we cannot fit enough of them to boost performance via parallelism as required by all bit-serial and bit-skipping designs.

Accordingly, **our second contribution** is *FPRaker*, a processing tile for *training* accelerators which exploits both bit-sparsity and out-of-bounds computations. *FPRaker* comprises several adder-tree based processing elements organized in a grid so that it can exploit data reuse both spatially and temporally. The processing elements multiply multiple value pairs concurrently accumulating their products. They process one of the input operands per multiplication as a series of signed powers of two hitherto referred to as *terms*. The conversion of that operand into powers of two is performed on the fly; all operands are stored in floating point form in memory. The processing elements take advantage of ineffectual work that stems either from mantissa bits that were zero or from out-of-bounds multiplications given the current accumulator value. The

tile owes its area efficiency to the following design choices: a) The processing element limits the range of powers-of-two that they can be processed simultaneously greatly reducing the cost of its shift-and-add components. b) A common exponent processing unit that is time-multiplexed among multiple processing elements. c) The power-of-two encoders are shared along the rows. d) Per processing element buffers reduce the effects of work imbalance across the processing elements. e) The PE implements a low cost mechanism for eliminating out-of-range intermediate values. Skipping out-of-bound intermediate values not only reduces the amount of work, but more importantly proves very effective in reducing the effect of cross-lane synchronization. Section 3.1 explains that a bit-parallel unit could only take advantage of these values to power-gate some of its component but not to also improve performance.

In more detail, *FPRaker* has the following characteristics: a) *FPRaker* is not an approximation and does not affect numerical accuracy. The results it produces adhere to the floating-point arithmetic used during training. b) It skips ineffectual operations that would result from zero mantissa bits and from out-of-range intermediate values. c) Despite individual MAC operations taking more than one cycle, *FPRaker*'s computational throughput is higher compared to conventional floating-point units; given that *FPRaker* processing elements are much smaller we can fit more of them in the same area. d) It naturally supports shorter mantissa lengths thus rewarding innovation in training methods with mixed or shorter datatypes [46, 53]. It does so while not requiring that the methods be universally applicable to all models. e) *FPRaker* allows us to choose which tensor input we wish to process serially per layer. This allows us to target those tensors that have more sparsity depending on the layer and the pass (forward or backward).

Our third contribution is a simple, low-overhead memory lossless encoding for floating-point values that relies on the value distribution that is typical in deep learning training. We observed that consecutive values across channels have similar values and thus exponents. Accordingly, we encode the exponents as deltas for groups of such values. We use this encoding when storing/reading values off chip, thus further reducing the cost of memory transfers.

We highlight the following experimental observations: a) While some neural networks naturally exhibit zero values (sparsity) during training, unless pruning is used, this is limited to the activations and the gradients. b) term-sparsity exists in all tensors including the weights and is much higher than sparsity. c) Compared to an accelerator using optimized bit-parallel FP32 processing elements and that can perform 4K bfloat16 [24, 54] MACs per cycle, a configuration that uses the same compute area to deploy *FPRaker* PEs is 1.5× faster and 1.4× more energy efficient. d) Performance benefits with *FPRaker* remain fairly stable throughout the training process for all three major operations. e) *FPRaker* can be used in conjunction with training methods that specify a different accumulator precision to be used per layer. There it can improve performance vs using an accumulator with a fixed width significand by 38% for ResNet18.

1.1 Studied Models

Table 1 lists the models studied. ResNet18-Q is a variant of ResNet18 trained using PACT [10] which quantizes both activations and weights down to 4b during training. ResNet50-S2 is a variant of ResNet50 trained using dynamic sparse reparameterization [39] which targets sparse learning that maintain high weight sparsity throughout the training process while achieving accuracy levels comparable to baseline training. SNLI performs natural language inference and comprises of fully-connected, LSTM-encoder, ReLU, and dropout layers. Image2Text is an encoder-decoder model for image-to-markup generation. We study three models of different tasks from MLPerf training benchmark: 1) Detectron2: an object detection model based on Mask R-CNN, 2) NCF: a model for collaborative filtering, and 3) Bert: a transformer-based model using attention. For our measurements we sample one randomly selected batch per epoch over as many epochs as necessary (at most 90) to train the network to its originally reported accuracy .

Table 1: Models Studied

Model	Application	Dataset
SqueezeNet 1.1	Image Classification	ImageNet [45]
VGG16	Image Classification	ImageNet
ResNet18-Q	Image Classification	ImageNet
ResNet50-S2	Image Classification	ImageNet
SNLI	Natural Language Infer.	SNLI Corpus [4]
Image2Text	Image-to-Text Conversion	im2latex-100k [55]
Detectron2	Object Detection	COCO [35]
NCF	Recommendation	ml-20m [23]
Bert	Language Translation	WMT17 [16]

2 INEFFECTUAL WORK IN TRAINING

This section motivates *FPRaker* by measuring the work reduction that is theoretically possible with two related approaches: 1) by removing all MACs where at least one of the operands are zero (value sparsity, or simply sparsity), and 2) by processing only the non-zero bits of the mantissa of one of the operands (bit sparsity).

The bulk of work during training is due to three major operations per layer:

$$\boldsymbol{Z = I \cdot W} \quad (1) \qquad \frac{\partial \boldsymbol{E}}{\partial \boldsymbol{I}} = \boldsymbol{W}^{\boldsymbol{T}} \cdot \frac{\partial \boldsymbol{E}}{\partial \boldsymbol{Z}} \quad (2) \qquad \frac{\partial \boldsymbol{E}}{\partial \boldsymbol{W}} = \boldsymbol{I} \cdot \frac{\partial \boldsymbol{E}}{\partial \boldsymbol{Z}} \quad (3)$$

For convolutional layers Eq. 1 describes the convolution of activations (I) and weights (W) that produces the output activations (Z) during forward propagation. There the output Z passes through an activation function before used as input for the next layer. Eq. 2 and 3 describe the calculation of the activation ($\frac{\partial E}{\partial I}$) and weight ($\frac{\partial E}{\partial W}$) gradients respectively in the backward propagation. Only the activation gradients are back-propagated across layers. The weight gradients update the layer's weights once per batch. For fully-connected layers the equations describe several matrix-vector operations. For other operations they describe vector operations or matrix-vector operations. For clarity, in the rest of this work we will refer to the gradients as G.

Figs 1 and 2 show the value, and term-sparsity respectively and for each of the three tensors (W, I, and G). Each value is weighted according to frequency of use. We use the term *term-sparsity* to signify that for these measurements the mantissa is first encoded into signed powers of two using Canonical-encoding which is a variation of Booth-encoding. This is because of the bit-serial processing of the mantissa.

The activations in the image classification networks exhibit sparsity exceeding 35% in all cases. This is expected since these networks use the ReLU activation function which clips negative values to

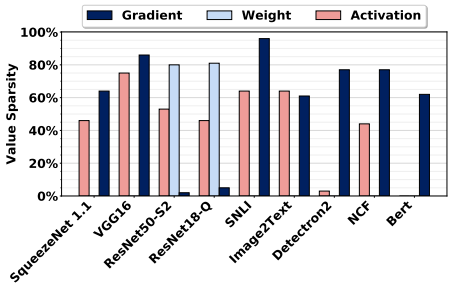

Figure 1: Value Sparsity

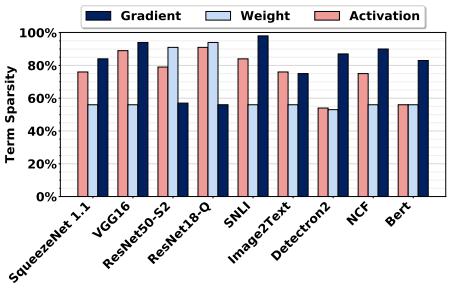

Figure 2: Term Sparsity

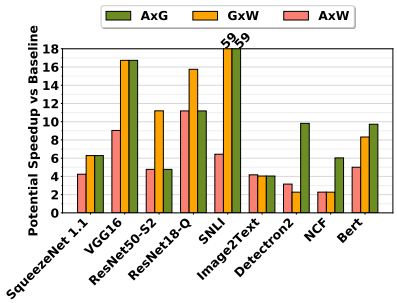

Figure 3: Performance improvement potential of exploiting term sparsity for the three training phases.

zero. However, Weight sparsity is typically low and only some of the classification models exhibit sparsity in their gradients. For the remaining models, however, such as those for natural language processing, value sparsity is very low for all three tensors. Regardless, since some models do exhibit sparsity it may be worthwhile to investigate whether it possible to exploit it during training. As explained in the introduction, this is a non-trivial task, as training is different than inference and exhibits dynamic sparsity patterns on all tensors and different computation structure during the backward pass.

Fig. 2 shows that all three tensors exhibit high term-sparsity for all models regardless of the target application. Given that term-sparsity is more prevalent than value sparsity in the rest of this work we investigate whether it is practically possible to exploit it during training. One such option is the *FPRaker* processing element which we present next.

Fig. 3 reports the ideal potential speedup due to reduction in the multiplication work through skipping the zero terms in the serial input. We calculate the potential speedup over the baseline as:

$$Potential\ speedup = \frac{\#MAC\ operations}{term\ sparsity \times \#MAC\ operations} \quad (4)$$

3 FPRAKER APPROACH

FPRaker's goal is to take advantage of bit sparsity in one of the operands used in the three operations performed during training (equations 1 through 3) all of which are composed of many MAC operations. We first explain how decomposing MAC operations into a series of shift-and-add operations can expose ineffectual work, providing us with the opportunity to save energy and time. Section 4.1 then describes the *FPRaker* processing element and Section 4.3 details the *FPRaker* tile.

3.1 Exposing Ineffectual Work

To expose ineffectual work during MAC operations we can decompose the operation into a series of "shift and add" operations, Let us first look at the multiplication. Let $A = 2^{A_e} \times A_m$ and $B = 2^{B_e} \times B_m$ be two values in floating point, both represented as an exponent (A_e and B_e) and a significant (A_m and B_m) which is normalized and includes the implied "1.". Conventional floating point units perform this multiplication in a single step (sign bits are XORed):

$$A \times B = 2^{A_e+B_e} \times (A_m \times B_m) = (A_m \times B_m) \ll (A_e + B_e) \quad (5)$$

By decomposing A_m into a series p of signed powers of two A_m^p where $A = \sum_p A_m^p$ and $A_m^p = \pm 2^i$, we can instead perform the multiplication as follows:

$$A \times B = (\sum_p A_m^p \times B_m) \ll (A_e + B_e) = \sum_p \pm B_m \ll (i + A_e + B_e) \quad (6)$$

For example, if A_m=1.0000001b, A_e=10b, B_m=1.1010011b and B_e=11b then we can perform $A \times B$ as two *shift-and-add* operations of $B_m \ll (10b+11b-0)$ and $B_m \ll (10b+11b-111b)$. A conventional multiplier would process all bits of A_m despite performing ineffectual work for the six bits that are zero.

This decomposition exposes further ineffectual work that conventional units perform as a result of the high dynamic range of values that floating point seeks to represent. Informally, some of the work done during the multiplication will result in values that will be *out-of-bounds* given the accumulator value. To understand why this is the case let us now consider not only the multiplication but also the accumulation. Let's assume that the product $A \times B$ will be accumulated into a running sum S and S_e is much larger than $A_e + B_e$. It will not possible to represent the sum $S + A \times B$ given the limited precision of the mantissa. In other cases, some of the "shift-and-add" operations would be guaranteed to fall outside the mantissa even when we consider the increased mantissa length used to perform rounding, i.e., partial swamping. A conventional pipelined MAC unit can at best power-gate the multiplier and accumulator after comparing the exponents and only when the *whole* multiplication result falls out of range. However, it cannot use this opportunity to reduce cycle count. By decomposing the multiplication into several simpler operations, we can terminate the operation in a single cycle given that we process the bits from the most to the least significand, and thus boost performance by initiating another MAC earlier. The same is true when processing multiple $A \times B$ products in parallel in an adder-tree processing element. A conventional adder-tree based MAC unit can potentially power-gate the multiplier and the adder tree branches corresponding to products that will be out-of-bounds. The cycle will still be consumed. As we explain in the next section, a shift-and-add based unit will be able to terminate such products in a single cycle and advance others in their place.

4 FPRAKER IMPLEMENTATION

4.1 FPRaker Processing Element

The *FPRaker* PE performs the multiplication of 8 Bfloat16 (A, B) value pairs, concurrently accumulating the result into an accumulator. The Bfloat16 format consists of a sign bit, followed by a biased 8b exponent, and a normalized 7b significand (mantissa). Fig. 4 shows a baseline of the *FPRaker* PE design which performs the computation in 3 blocks: exponent, reduction, and accumulation.

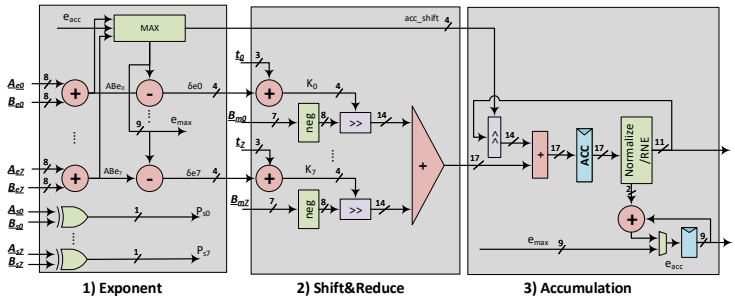

Figure 4: *FPRaker* PE: Baseline Design.

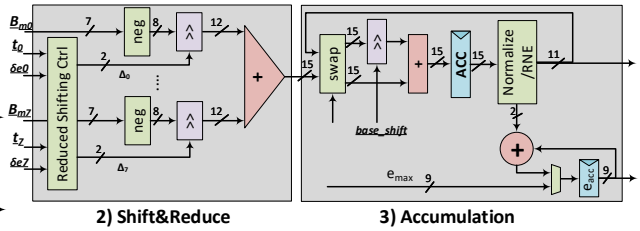

Figure 5: *FPRaker*PE: Modified Design.

We describe an implementation where the 3 blocks are performed in a single cycle. We will build upon this design and modify it to construct a more area efficient tile comprising several of these PEs. Recall that the significands of each of the A operands are converted on-the-fly into a series of *terms* (signed powers of two) using canonical encoding, e.g. A=(1.1110000) is encoded as $(+2^{+1}, -2^{-4})$. This encoding occurs just before the input to the PE. All values stay in bfloat16 while in memory. The PE will process the A values term-serially. The accumulator has an extended 13b significand; 1b for the leading 1 (hidden), 9b for extended precision following the chunk-based accumulation scheme as suggested by Sakr et al., [47] with a chunk-size of 64, plus 3b for rounding to nearest even. It has 3 additional integer bits following the hidden bit so that it can fit the worst case carry out from accumulating 8 products. In total the accumulator has 16b, 4 integer, and 12 fractional.

The PE accepts 8 8-bit A exponents $A_{e0}, ..., A_{e7}$, their corresponding 8 3-bit significand *terms* $t_0, ..., t_7$ (after canonical encoding) and signs bits $A_{s0}, ..., A_{s7}$, along with 8 8-bit B exponents $B_{e0}, ..., B_{e7}$, their significands $B_{m0}, ..., B_{m7}$ (as-is) and their sign bits $B_{s0}, ..., B_{s7}$ as shown in Fig. 4.

Block 1 — Exponent: Processing a new set of 8 value pairs starts first at the exponent block. This block adds the A and B exponents in pairs to produce the exponents ABe_i for the corresponding products. A comparator tree (MAX) takes these product exponents and the exponent of the accumulator and calculates the maximum e_{max}. The maximum exponent is used to align all products so that they can be summed correctly. To determine the proper alignment per product the block subtracts all product exponents from e_{max} calculating the alignment offsets δe_i. The maximum exponent is used to also discard terms that will fall out-of-bounds when accumulated. The PE will skip any terms who fall outside the $e_{max}-12$ range. The block is invoked only *once* per new set of value pairs, before any terms are generated, and regardless of how many terms end up being generated. Accordingly, the minimum effective number of cycles for processing the 8 MACs will be 1 cycle regardless of value (the blocks can be pipelined, and since there are no data dependencies, the pipeline can be kept full). In case one of the resulting products has an exponent larger than the current accumulator exponent, the accumulator will be shifted accordingly prior to accumulation (*acc_shift* signal).

Block 2 — Shift&Reduce: Since multiplication with a term amounts to shifting, this block calculates the number of bits by which each B significand will have to be shifted by prior to accumulation. These are the 4-bit terms $K_0, ..., K_7$. To calculate K_i we add the product exponent deltas (δe_i) to the corresponding A term t_i. The A sign bits are XORed with their corresponding B sign bits to determine the signs of the products $P_{s0}, ..., P_{s7}$. The B significands are complemented according to their corresponding product signs, and then shifted using the offsets $K_0, ..., K_7$. The PE uses a shifter per B significand to implement the multiplication. In contrast, a conventional floating point unit would require shifters at the output of the multiplier. Thus *FPRaker* PE effectively completely eliminates the cost of the multipliers.

Bits that are shifted out of the accumulator range from each B_m operand are rounded using round-to-nearest-even (RNE) approach. An adder tree reduces the 8 B_m operands into a single partial sum as shown in Fig. 4(2).

Block 3 — Accumulation: The resulting partial sum from step 2 is added to the correctly aligned value of the accumulator register. In each accumulation step, the accumulator register is normalized and rounded using the rounding-to-nearest-even (RNE) scheme. The normalization block updates the accumulator exponent as shown in Fig. 4(3). When the accumulator value is read out, it is converted to bfloat16 by extracting only 7b for the significand.

In the worst case two K_i offsets may differ by up to 12 since our accumulator has 12 fractional bits. This means that the baseline PE requires relatively large shifters and an accumulator tree that accepts wide inputs. Specifically, the PE requires shifters that can shift up to 12 positions a value that is 8b (7b significant + hidden bit). Had this been integer arithmetic we would need to accumulate 12+8=20b wide. However, since this is a floating point unit we will be accumulating only the 14 most significant bits (1b hidden, 12b fractional and the sign). Any bits falling below this range will be included in the sticky bit which is the least significant bit of each input operand.

It is possible to further reduce this cost by taking advantage of the expected distribution of the exponents. Fig. 6 shows, for example, the distribution of exponents for a layer of ResNet34. The vast majority of the exponents of the inputs, the weights and the output gradients lie within a narrow range. This suggests that in the common case the exponent deltas will be relatively small. In addition, the MSBs of the activations are guaranteed to be one (given denormals are not supported [24]). This indicates that very often the $K_0, ..., K_7$ offsets would lie within a narrow range. We take advantage of this behavior to reduce the PE area. In our preferred configuration we limit the maximum difference in among the K_i offsets that can be handled in a *single cycle* to be up to 3. As a result, the shifters need to support shifting by up to 3b and the adder tree now needs to process 12b inputs (1b hidden, 7b+3b significant, and the sign bit). A shared *single* shifter ($base\_shift$) after the adder tree serves a dual purpose: First, it aligns the adder tree's output and the accumulator properly. Second, it allows the PE to

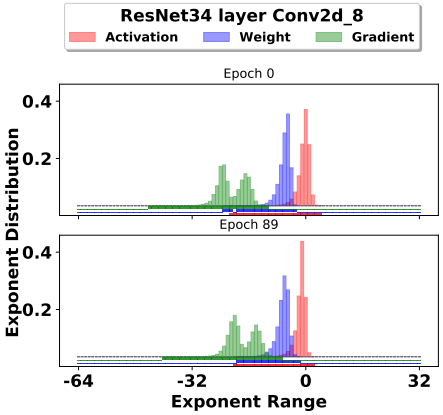

Figure 6: Exponent histogram of layer *Conv2d_8* in epochs 0 and 89 of training ResNet34 on ImageNet. The figure shows only the utilized part of the full range [-127:128] of an 8b exponent.

skip over longer than 3b distances in the input term stream. This is useful, when the next set of terms are at a distance longer than 3b vs. the current ones. Fig. 5 shows the modified PE. Each PE has a control unit to generate the modified terms Δ_i and a $valid_{\Delta i}$ signal indicating whether the lane can process its term at the current cycle.

Skipping out-of-bounds terms turns out to be surprisingly inexpensive. The control unit uses a comparator per lane to check if its current K term lies within a threshold with the value of the accumulator precision (comparators are optimized by the synthesis tool for comparing with a constant) and feeds back an OB_i signal to its corresponding term encoder indicating that any subsequent term coming from the same input pair is guaranteed to be ineffectual (out-of-bound) given the current e_{acc} value. Hence, *FPRaker*can boost its performance and energy-efficiency by skipping the processing of the subsequent out-of-bound terms. The OB_i signals of a certain lane across the PEs of the same tile column are synchronized together. The threshold is currently set according to [47] which ensures models converge within 0.5% of the FP32 training accuracy on ImageNet dataset. However, the threshold can be controlled effectively implementing a dynamic bit-width accumulator which can boost performance by increasing the number of skipped "out-of-bounds" bits, an option we do not investigate further.

4.2 Sharing the Exponent Block

In the common case, processing a group of A values will require multiple cycles since some of them will be converted into multiple terms. During that time the exponent block inputs will not change. To further reduce area we can take advantage of this expected behavior and share the exponent block across multiple PEs. The decision of how many PEs to share an exponent block over can be based on the expected bit-sparsity. The lower the bit-sparsity the longer the processing time per PE, the less often it will need new exponents, and the more the PEs can share the exponent block. Since the studied models are highly sparse, sharing one exponent block per two PEs proved best. Figure 7 shows the modified design. The unit as a whole accepts as input one set of 8 A inputs and two sets of B inputs, B and B'. The exponent block can process one of

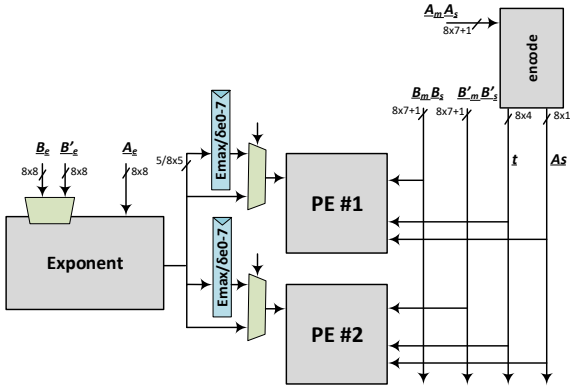

Figure 7: PEs sharing the exponent block.

(A, B) or (A, B') at a time. During the cycle when it processes (A, B) the multiplexer for PE#1 passes on the e_{max} and exponent deltas directly to the PE. Simultaneously, these values will be latched into the registers in front of the PE so that they remain constant while the PE processes all terms of input A. When the exponent block processes (A, B') the aforementioned process proceeds with PE#2. With this arrangement both PEs must finish processing all A terms before they can proceed to process another set of A values. Since the exponent block is shared, each set of 8 A values will take at least 2 cycles to be processed (even if it contains zero terms).

4.3 FPRaker Tile

By utilizing per PE buffers it is possible to exploit data reuse temporally. To exploit data reuse spatially we can arrange several PEs into a tile. Fig. 8 shows an example of a 2×2 tile of PEs and each PE performs 8 MAC operations in parallel. Each pair of PEs per column shares an exponent block as previously described. The B and B' inputs are shared across PEs in the same row. For example, during the forward pass we can have different filters being processed by each row and different windows processed across the columns. Since the B and B' inputs are shared all columns would have to wait for the column with the most A_i terms to finish before advancing to the next set of B and B' inputs. To reduce these stalls the tile introduces per B and B' buffers. N buffers per PE allow the columns be at most N sets of values ahead.

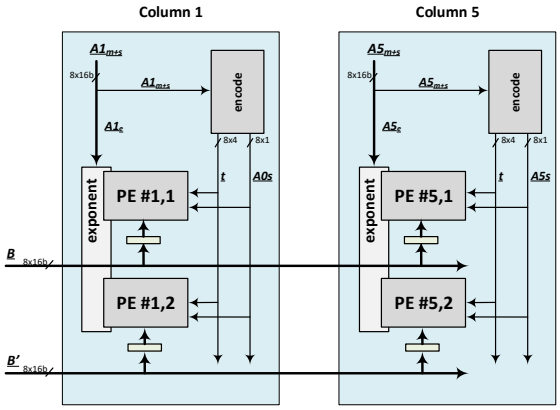

Figure 8: A 2×2 PE *FPRaker* Tile.

4.4 Numerical Fidelity and Convergence

A concern with any hardware or software implementation of floating-point arithmetic is whether it maintains the numerical fidelity necessary by the target algorithm. In our case, the question is whether *FPRaker* affects training convergence. Section 6.5 demonstrates experimentally that training accuracy and epoch count are unaffected by *FPRaker*. This should not come as a surprise. Specifically, for any implementation there are two major considerations: 1) whether *individual* operations meet some relevant floating-point standard, typically a version of the IEEE 754 [1], and 2) whether any reordering of the *sequence* of operations used may "break" the algorithm itself. Such concerns are universal applying to all hardware implementations, be it CPUs, GPUs, or accelerators, to compilers since they schedule operations and perform other optimizations, and even at the algorithmic level. Goldberg presents an excellent treatment on the subject [18].

Rather than using individual MACs, *FPRaker* implements a group MAC, and then instead of processing all mantissa bits simultaneously, it does so serially. Its output adheres to the floating-point standard given the specific grouping of operations. Often the result's accuracy exceeds the minimum set by the standard since bits that would otherwise be discarded when performing each MAC operation individually, may now survive as small sums for LSb accumulate together (this occurs when many small numbers are accumulated first together prior to being added to a larger running sum). There have been other cases where fusing multiple operations together resulted in more accurate functional units. For example, this includes the Fused multiply-accumulate units unveiled by IBM in the early 90s [26] . Moreover, recent work has shown that performing group MAC is key to reducing floating-point mantissas during training [46].

Still, one may wonder, however, whether the use of group MAC operations in itself violates numerical integrity as it forces a specific ordering of the operations within each group. We remind the reader that algorithm have to content with nearly identical challenges even on commodity hardware. Consider for example the effects of the hardware scheduler, warp configuration, and of the compiler for modern GPUs. Any rearrangement of the MAC operations may affect the final result. While instructions within the same warp are guaranteed to execute in program order (assuming no control flow), warp size is not part of the language specification. Accordingly, unless the dataflow constraints all MAC operations contributing to the same output neuron to one thread, the execution order.

4.5 Exponent Base-Delta Compression

The narrow value distribution shown in Fig. 6 motivated us to study the spatial correlation of values. We found that consecutive values across all dimensions (channel, H, or W). Accordingly, we store the exponents of groups of neighboring values as delta from a common base exponent. This is true for the activations, the weights and the output gradients. Similar values in floating-point have similar exponents, a property which we can exploit through a base-delta compression scheme [42]. In our experiments, we block values channel-wise (we present evidence, however, that the value correlation persists along the H dimension as well) into groups of 32 values each, where the exponent of the first value in the group is the base and we compute the delta exponent for the rest of the values in the group relative to it as shown in Fig. 9. The bit-width (δ) of the delta exponents is dynamically determined per group and is set to the maximum precision (P) of the resulting delta exponents per group similar to the approach of Delmas et al. [33]. The delta exponent bit-width (3b) is attached to the header of each group as metadata.

Fig. 10 shows the total, normalized exponent footprint after the base-delta compression. Our technique is effective for both channel-wise and spatial (H dimension shown) dataflows. We use this compression scheme to reduce the off-chip memory bandwidth. Values are compressed at the output of each layer and before writing them off-chip, and they are decompressed when they are read back on-chip.

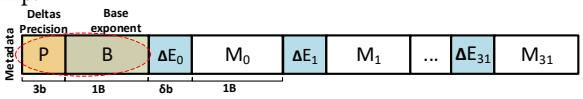

Figure 9: A 32-value group with exponent base-delta compression

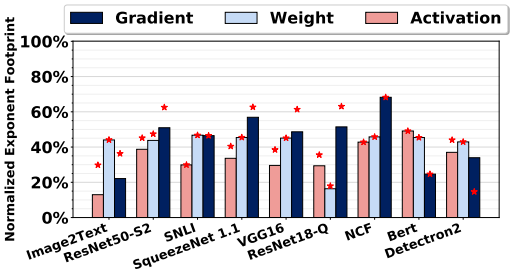

Figure 10: Memory savings due to exponent base-delta compression. Bars and markers represent compression channel-wise and spatial-wise, respectively.

4.6 Data Supply

Focusing solely on computation is insufficient. Data transfers often dominate energy consumption. Accordingly, it is essential to consider what the memory hierarchy needs to do to keep the execution units busy. A challenge with training is that while it processes three arrays I, W and G the order in which their elements are grouped differs across the three major computations (Eq. 1 through 3). However, we can rearrange the arrays as they are read from off-chip. Accordingly, we store the arrays in memory using a container of "square" of 32×32 bfloat16 values. This size matches well the typical row sizes of DDR4 memories and allows us to achieve high bandwidth when reading values from off-chip. A container includes values from coordinates (c, r, k) (channel, row, column) to $(c+31, r, k+31)$ where c and k are divisible by 32 (padding is used as necessary). Containers are stored in channel, column, row order. When read from off-chip memory the container values are stored in the exact same order on the multibanked on-chip buffers. The tiles can then access data directly reading 8 bfloat16 values per access. The weights and the activation gradients, however, need to be processed in the transpose order for some operations. For this purpose we incorporate transposer units on-chip. A transposer reads in 8 blocks of 8 bfloat16 values from the on-chip memories. Each of these 8 reads uses 8-value wide reads as mentioned above and the blocks are written as rows in an internal to the transposer buffer. Collectively these blocks form an 8×8 block of values. The transposer can

Table 2: Baseline and *FPRaker* configurations.

	FPRaker	Baseline
Tile Configuration	8 × 8	8 × 8
Tiles	36	8
Total PEs	2304	512
Multipliers/PE	8	8 BFLOAT16
MACs/cycle	-	4096
Scratchpads	2KB each	
Global Buffer	4MB × 9 banks	
Off-chip DRAM Memory	16GB 4-channel LPDDR4-3200	

read out 8 blocks of 8 values each and send those to the processing units. Each of these blocks however is read out as a column from its internal buffer. This effectively transposes the 8 × 8 value group.

5 METHODOLOGY

A custom cycle-accurate simulator was developed to model the execution time of *FPRaker* and of the baseline architecture. Besides modeling timing behavior the simulator also models value transfers and computation in time faithfully and checks the produced values for correctness against the golden values. The simulator was validated with microbenchmarking. For area and power analysis, both *FPRaker* and the baseline designs were implemented in Verilog and synthesized using Synopsys' Design Compiler [32] for 600 MHz clock frequency with a 65nm TSMC technology (due to licensing restrictions we cannot get access to a better technology) and with a commercial library for the given technology. We use Cadence Innovus [5] for layout generation. We use Intel's PSG ModelSim to generate data-driven activity factors which we feed to Innovus to estimate the power. The baseline MAC unit was optimized for area, energy and latency. Generally, it is not possible to optimize for all three, however, in the case of MAC units, it is possible [17]. We use an efficient bit-parallel fused MAC unit as the baseline PE. The constituent multipliers are both area and latency efficient, and are taken from the DesignWare IP library developed by Synopsys. Further, we optimize the baseline units for deep learning training by reducing the precision of its I/O operands to bfloat16 and accumulating in reduced precision with chunk-based accumulation similar to FPRaker units. The area and energy consumption of the on-chip SRAM Global Buffer (GB) is divided into activation, weight, and gradient memories which were modeled using CACTI [25]. The Global Buffer has an odd number of banks to reduce bank conflicts for layers with a stride greater than one. To estimate the latency and energy consumption of the off-chip DRAM memory we use the model provided by Micron [38]. The configurations for both *FPRaker* and the baseline are shown in Table 2.

To evaluate our accelerator, we collected traces for one random mini-batch during the forward and backward pass in each epoch of training. All models were trained long enough to attain the maximum top-1 accuracy as reported by the original authors. To collect the traces, we trained each model on a NVIDIA RTX 2080 Ti GPU and stored all of the inputs and outputs for each layer using Pytorch Forward and Backward hooks. For BERT we traced BERT-base and the fine-tuning training for a GLUE task. The simulator uses the traces to model execution time and collects activity statistics so that energy can be modeled.

6 EVALUATION

Section 6.2 evaluates the performance of *FPRaker* over the baseline architecture. Section 6.3 reports the energy efficiency. Section 6.6 shows *FPRaker* performance sensitivity with increasing the number of rows per tile. Section 6.7 shows *FPRaker* performance gain with per layer precision profiling.

6.1 Area

Since *FPRaker* processes one of the inputs term-serially a single *FPRaker* processing engine can never outperform a conventional PE that processes the same number of inputs. *FPRaker* relies on parallelism to extract more performance. This is only possible if we can afford to use more *FPRaker* units than conventional units. One approach is to use an iso-compute area constraint. That is to determine how many *FPRaker* tiles we can fit in the same area for a baseline tile. For this we take into account only the compute cores as associated logic and not the scratchpads.

The conventional PE we compared against processes concurrently 8 bfloat16 value pairs accumulating their sum. We can include buffers for the inputs (A and B) and the outputs to exploit data reuse temporally. We can also organize multiple PEs in a grid sharing buffers and inputs across rows and columns to also exploit reuse spatially. Both *FPRaker* and the baseline are configured to have scaled-up GPU Tensor-Core-like tiles that perform 8 × 8 vector-matrix multiplication where 64 PEs are organized in a 8 × 8 grid and each PE performs 8 MAC operations in parallel.

Post layout, and taking into account only the compute area, an *FPRaker* tile occupies 22% the area vs. the baseline tile. Table 3 reports the corresponding area and power per tile. Accordingly, to perform an iso-compute-area comparison, we configure the baseline accelerator to have 8 tiles and *FPRaker* with 36 tiles. The area for the on-chip SRAM global buffer is $344mm^2$, $93.6mm^2$, and $334mm^2$ for the activations, weights, and gradients, respectively.

Table 3: Breakdown of the area and power consumption per tile of *FPRaker* vs. Baseline.

Area [μm^2]				
	PE Array	**Term Encoders**	**Total**	**Normalized**
FPRaker	304,118	12,950	317,068	0.22×
Baseline	1,421,579	N/A	1,421,579	1×
Power [mW]				
FPRaker	104	5.5	109.5	0.23×
Baseline	475	N/A	475	1×
Energy Efficiency Normalized to Baseline				
FPRaker			1.75×	

6.2 Execution Time

Figure 11 shows the performance breakdown of *FPRaker*due to the base-delta compression (BDC), and skipping zero and out-of-bound (OB) terms relative to the baseline. On average, *FPRaker* outperforms the baseline by 1.5× (skipping zero terms: +9%, BDC: +5.8%, skipping out-of-bound terms: +35.2%). From the studied convolution-based models, ResNet18-Q benefits the most from *FPRaker* where the performance improves by 2.04× over the baseline. Training for this network incorporates PACT quantization and as a result most of the activations and weights throughout the training process can fit in 4b or less. This translates into high term sparsity which *FPRaker* exploits. This result demonstrates that *FPRaker* can deliver benefits with specialized quantization methods without requiring that the hardware be also specialized for this purpose.

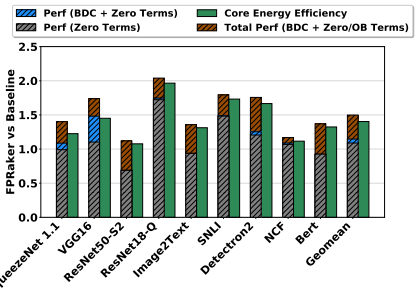

Figure 11: iso-compute-area performance and energy-efficiency comparison between *FPRaker*and the baseline.

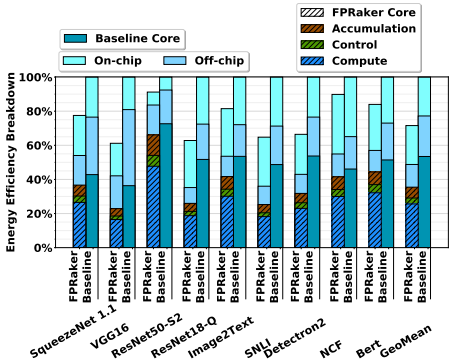

Figure 12: Overall Energy Efficiency of *FPRaker*vs. Baseline.

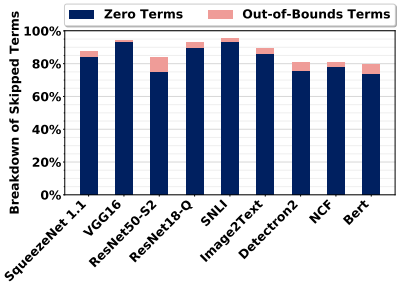

Figure 13: Breakdown of skipped terms by *FPRaker*.

SNLI, NCF, and Bert are dominated by fully connected layers. while in fully-connected layers there is no weight reuse among different output activations, training can take advantage of batching to maximize weight reuse across multiple inputs (e.g., words) of the same input sentence which results in higher utilization of the tile PEs. Speedups follow bit sparsity. For example, *FPRaker* achieves a speedup of 1.8× over the baseline for SNLI due its high bit sparsity.

Table 3 reports that the total power of an FPRaker tile with 8×8 PEs is 109.5mW, while the total power of a baseline tile is 475mW. Accordingly, the FPRaker uses 3.8% more power than the baseline while being 1.5x times faster. There are multiple adjustments that can be made to further improve power efficiency such as reducing the frequency or the number of tiles. Compared to the baseline, a configuration with 28 FPRaker tiles is 20% more power-efficient, 16.6% faster, and 31% more energy-efficient.

6.3 Energy Efficiency

Figure 11 reports total energy efficiency of *FPRaker* over the baseline architecture per model. On average, *FPRaker* is 1.4× more energy efficient compared to the baseline considering only the core logic and 1.36× more energy efficient when everything is taken into account. The energy-efficiency improvements follow closely the performance benefits. For example, benefits are higher at around 1.7× for SNLI and Detectron2. The quantization in ResNet18-Q boosts the core logic energy efficiency to as high as 1.97×. Fig. 12 shows the energy consumed by *FPRaker* normalized to the baseline as a breakdown across three main components: core logic, off-chip and on-chip data transfers. Fig. 12 further breaks down *FPRaker*core into "Compute" (PE stages 1 and 2), "Control" (PE control units and shared term encoders), and "Accumulation" (PE stage 3). *FPRaker* along with the exponent base-delta compression reduce the energy consumption of the core logic and off-chip memory significantly.

6.4 Performance Analysis

Skipped Terms: Figure 13 shows a breakdown of the terms *FPRaker* skips. There are two cases: 1) skipping zero terms, and 2) skipping non-zero terms that are out-of-bounds due to the limited precision of the floating-point representation.

Skipping out-of-bounds terms increases term sparsity for ResNet50-S2 and Detectron2 by around 10% and 5.1%, respectively. Networks with high sparsity (zero values) such as VGG16 and SNLI benefit the least from skipping out-of-bounds terms with the majority of term sparsity coming from zero terms. This is because there are few terms to start with. For ResNet18-Q, most benefits come from skipping zero terms as the activations and weights are effectively quantized to 4b values. However, as Figure 11 showed, skipping out-of-bound terms improves performance much more than the fraction of terms skipped would suggest. Recall, that all lanes must wait for the slowest one to finish processing amplifying the effect on performance across all lanes. In the worse case, all other 7 lanes are waiting for a single one to finish, wasting 7 execute slow. Skipping out-of-bound terms reduces this synchronization overhead.

Computation Phase: Figure 14 reports speedup for each of the 3 phases of training: the $A \times W$ in forward propagation, and the $A \times G$ and the $G \times W$ to calculate the weight and input gradients in the backpropagation, respectively. *FPRaker* consistently outperforms the baseline for all three phases. The speedup depends on the amount of term sparsity, and the value distribution of A, W, and G across models, layers, and training phases. The less terms a value has the higher the potential for *FPRaker* to improve performance. However, due to the limited shifting that the *FPRaker* PE can perform per cycle (up to 3 positions) how terms are distributed within a value impacts the number of cycles needed to process it. This behavior applies across lanes to the same PE and across PEs in the same tile. In general, the set of values that are processed concurrently will translate into a specific term sparsity pattern. *FPRaker* favors patterns where the terms are close to each other numerically.

Where Cycles Go: Figure 15 reports a breakdown of PE lane utilization and highlights performance bottlenecks. Stalls occur due to: 1) inter-PE synchronization, 2) intra-PE synchronization, and 3) Sharing the exponent block. Intra-PE synchronization stalls can happen in two scenarios: a) imbalance in the number of terms assigned for each lane within a PE due to uneven distribution of the term sparsity in the model which results in idle lanes waiting for the slowest lane to finish execution ("no terms"), b) high span across consecutive terms within a lane which cause stalls due to the limited per cycle shift range of the PE ("shift range").

Stalls due to sharing the exponent block are rare. The more bit sparsity a model has, the higher the pressure on the exponent block and the chance that it will not be able to keep up. Exponent stalls are

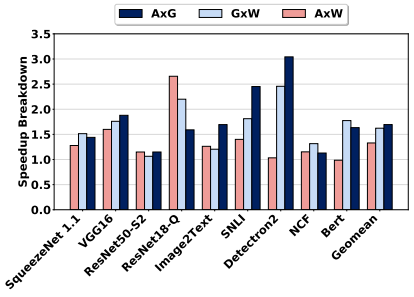

Figure 14: Breakdown of *FPRaker* speedup over the baseline.

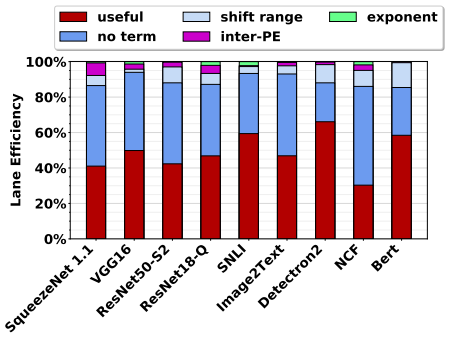

Figure 15: Breakdown of execution cycles of *FPRaker*.

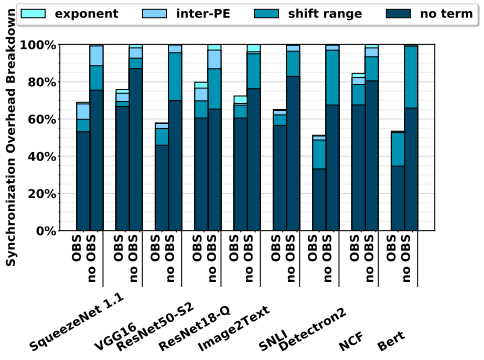

Figure 16: Effect of out-of-bound terms skipping (OBS) on the synchronization overhead.

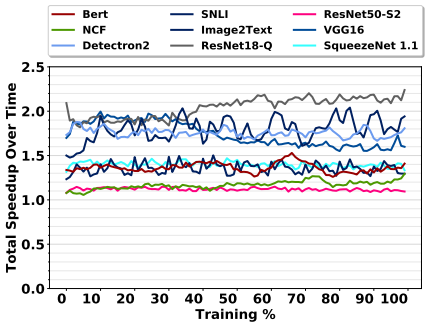

Figure 17: Speedup of *FPRaker* vs. the baseline over time.

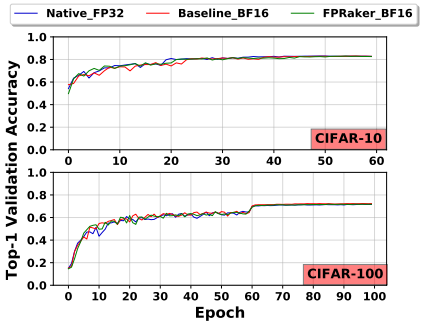

Figure 18: Top-1 validation accuracy of training ResNet18 by emulating the FPRaker processing in PlaidML.

noticeable for ResNet18-Q since the values there are effectively 4b. SNLI stalls are due to its high bit sparsity. However, there are other types of stalls that reduce pressure on the shared exponent block. First are stalls due to the limited per cycle shifting ability of the PEs. These are relatively few thus demonstrating that this technique presents a good performance vs. area trade-off. Second are stalls due to cross-lane term imbalance. These are the highest cause of PE underutilization; 32.8% on average, and at most 55% for NCF. Term imbalance is lowered if we reduce the number of lanes per PE or if we add weight buffers to allow faster lanes to proceed with the next set of weights albeit with a higher area overhead. However, doing so would increase the cost of the PE. This investigation is left for future work. Third are stalls due to inter-PE synchronization which are also rare. The ability for PE columns to run ahead by just one set sufficiently hides these stalls. Figure 16 shows the benefit of skipping out-of-bound terms in reducing the synchronization overheads (an average of 30.3% overall reduction) by improving the load balancing across the PE lanes.

Performance over Time: Figure 17 shows speedup with *FPRaker* over the baseline throughout the training process. There are three trends: For VGG16 speedup is higher for the first 30 epochs after which it declines by around 15% and plateaus. For ResNet18-Q, the speedup increases after epoch 30 by around 12.5% and stabilizes. This can be attributed to the PACT clipping hyperparameter being optimized to quantize activations and weights within 4-bits or below. For the rest of the networks, speeds ups remain stable throughout the training process. Overall, the measurements show that performance of *FPRaker* is robust and that it delivers performance improvements across all training epochs.

6.5 Accuracy Study

To study the effect of training with FPRaker on accuracy, we emulated the bit-serial processing of FPRaker PE during end-to-end training in PlaidML [43] which is a machine learning framework based on an OpenCL compiler at the backend. We force PlaidML to use the mad() function for every multiply-add during training. We override the mad() function with our implementation to emulate the processing of the FPRaker PE. We trained ResNet18 on CIFAR-10 and CIFAR-100 datasets as shown in Fig. 18. The blue line shows the top-1 validation accuracy for training natively in PlaidML with FP32 precision. The baseline performs bit-parallel MAC with I/O operands precision in bfloat16 which is known to converge and supported in the industry, e.g. in Google's TPU. The figure shows that both the baseline and FPRaker emulated versions converge at epoch 60 for both datasets with accuracy difference within 0.1% relative to the native training version. This is expected since FPRaker skips only ineffectual work, i.e., work which does not affect final result in the baseline MAC processing.

6.6 Effect of Tile Organization

As Figure 19 shows, increasing the rows per tile reduces performance by 6% on average. This reduction in performance is due to synchronization among a larger number of PEs per column. When the number of rows increases, more PEs share the same set of A values. An A value that has more terms than the others will now affect a larger number of PE which will have to wait to finish processing. Since each PE processes a different combination of input vectors, each can be affected differently by intra-PE stalls such as

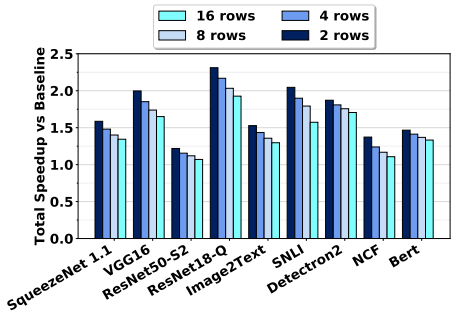

Figure 19: Speedup of *FPRaker* vs. the baseline with varying the number of rows per tile.

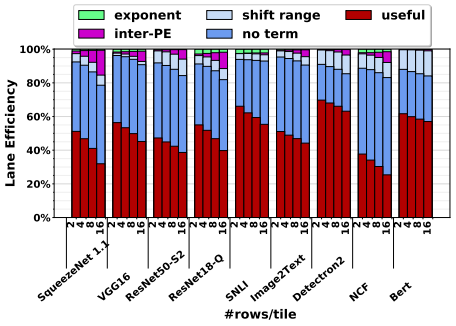

Figure 20: Varying the number of rows: Breakdown of Cycles.

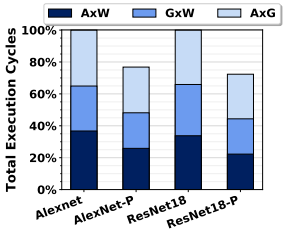

Figure 21: Performance of *FPRaker* with per layer profiled accumulator width [46] vs. fixed accumulator width.

"no term" stalls or "limited shifting" stalls. Figure 20 shows a breakdown of where time goes in each configuration. It can be seen that the stalls for the inter-PE synchronization increase and so do those for stalling for other lanes ("no term").

6.7 Per Layer Accumulator Width Profiling

Conventionally, training uses bfloat16 for all computations. As we noted in the introduction, there have been proposal for using mixed datatype arithmetic where some of the computations used fixed-point instead [11, 14, 37, 40]. Sakr et. al, propose to use floating-point where however the number of bits used by the mantissa varies per operation and per layer [46]. We use the suggested mantissa precisions while training AlexNet and ResNet18 on ImageNet. Figure 21 shows the performance of *FPRaker* following this approach. *FPRaker* can dynamically take advantage of the variable accumulator width per layer to skip the ineffectual terms mapping outside the accumulator boosting overall performance. Training ResNet18 on ImageNet with per layer profiled accumulator width boosts the speedup of *FPRaker* by 1.51×, 1.45× and 1.22× for $A \times W$, $G \times W$ and $A \times G$, respectively achieving an overall speedup of 1.56× over the baseline compared to 1.13× that is possible when training with a fixed accumulator width. Adjusting the mantissa length while using a bfloat16 container manifests itself a suffix of zero bits in the mantissa.

7 RELATED WORK

Compared to inference, training performs more computation and requires a larger volume of data to be kept around. While in inference activations and weights are used only once during training they are used twice and more importantly the dataflow is different between these two uses. This challenges several optimizations that are otherwise possible in inference. For example, inference accelerators targeting sparsity such as Cambricon-X [59] or SCNN [41] prepack values in memory according to pre-determined dataflow to eliminate zero values. Unfortunately, this is not directly compatible with training as the order in which values will be accessed during backpropagation needs to be different than that during forward pass (inference). At the microarchitectural level, *FPRaker* is a floating-point PE where the processing of values differs significantly from that of fixed-point values. The similarity between *FPRaker* and Bit-Pragmatic is limited in the use of the 2-stage shifting technique which we adapted to reduce the area of our adder tree. Laconic uses a unique processing element which is designed for fixed-point arithmetic only. During inference most values are of small magnitude and thus exhibit zero-bit prefixes. Moreover, the weights are static and activations exhibit significant sparsity. Training uses floating-point values with aligned mantissas where low magnitude does not translate into zero prefixes. Moreover, the mantissas exhibit noise in their least significant bits. In addition, floating point values have exponents which are non-existent in fixed-point. Finally, training uses randomly initialized values for weights and activations and also processes gradients.

Bit-serial arithmetic circuits have long been used in embedded applications such as in signal processing with the emphasis being on reducing cost at the expense of performance. The closest related designs are the single multiplier of Shinde and Salankar [51] and the single MAC of Chau et al. [7] which targets fault tolerance. Both use a multiplier stage, however, *FPRaker* is a SIMD MAC that processes Booth terms across multiple value pairs eliminating the need for a multiplier stage and bares little similarity at the microarchitectural level with the aforementioned designs.

8 CONCLUSION

While we evaluated *FPRaker* for training, it can naturally also be used for inference. While many neural network models can use fixed-point arithmetic there are models that still require floating-point. For example, these include models that process language or recommendation systems. Of course, whether *FPRaker* will provide benefits for inference with such models needs to be demonstrated.

FPRaker opens new avenues of research in efficient precision training. Different precisions can be assigned to each layer during training depending on the layer's sensitivity to quantization. Further, training can start with lower precision and increase the precision per epoch near conversion. FPRaker can adapt dynamically to different precisions and rewards innovations in training algorithms by boosting performance and energy efficiency.

ACKNOWLEDGMENTS

This work was supported by an NSERC Discovery Grant and the NSERC COHESA Strategic Research Network. The University of Toronto maintains all rights to the technologies described.

REFERENCES

[1] [n. d.]. ANSI/IEEE Std 754-2019 http://754r.ucbtest.org. https://754r.ucbtest.org/background/

[2] Jorge Albericio, Alberto Delmás, Patrick Judd, Sayeh Sharify, Gerard O'Leary, Roman Genov, and Andreas Moshovos. 2017. Bit-pragmatic Deep Neural Network Computing. In *Intl' Symp. on Microarchitecture.*

[3] Jorge Albericio, Patrick Judd, Tayler Hetherington, Tor Aamodt, Natalie Enright Jerger, and Andreas Moshovos. 2016. CNVLUTIN: Ineffectual-Neuron-Free Deep Neural Network Computing. In *Intl' Symp. on Computer Architecture*.

[4] Samuel R. Bowman, Gabor Angeli, Christopher Potts, and Christopher D. Manning. 2015. A large annotated corpus for learning natural language inference. In *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Lisbon, Portugal, 632–642. https://doi.org/10.18653/v1/D15-1075

[5] Cadence. [n. d.]. Innovus Implementation System. https://www.cadence.com/content/cadence-www/global/en_US/home/tools/digital-design-and-signoff/hierarchical-design-and-floorplanning/innovus-implementation-system.html.

[6] Francisco M. Castro, Manuel J. Marín-Jiménez, Nicolás Guil, Cordelia Schmid, and Karteek Alahari. 2018. End-to-End Incremental Learning. In *Computer Vision - ECCV 2018 - 15th European Conference, Munich, Germany, September 8-14, 2018, Proceedings, Part XII*. 241–257. https://doi.org/10.1007/978-3-030-01258-8_15

[7] P Chau, K. Chew, and W. Ki. 1987. A Bit-Serial Floating-Point Complex Multiplier-Accumulator for Fault-Tolerant Digital Signal Processing Arrays. In *IEEE International Conference on Acoustics, Speech, and Signal Processing*. IEEE, 483–486.

[8] Yu-Hsin Chen, Tien-Ju Yang, Joel S. Emer, and Vivienne Sze. 2019. Eyeriss v2: A Flexible Accelerator for Emerging Deep Neural Networks on Mobile Devices. *IEEE J. Emerg. Sel. Topics Circuits Syst.* 9, 2 (2019), 292–308. https://doi.org/10.1109/JETCAS.2019.2910232

[9] Yu-Hsin Chen, Joel Emer, and Vivienne Sze. 2016. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. In *ACM SIGARCH Computer Architecture News*, Vol. 44. IEEE Press, 367–379.

[10] Jungwook Choi, Zhuo Wang, Swagath Venkataramani, Pierce I-Jen Chuang, Vijayalakshmi Srinivasan, and Kailash Gopalakrishnan. 2018. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085* (2018).

[11] Dipankar Das, Naveen Mellempudi, Dheevatsa Mudigere, Dhiraj D. Kalamkar, Sasikanth Avancha, Kunal Banerjee, Srinivas Sridharan, Karthik Vaidyanathan, Bharat Kaul, Evangelos Georganas, Alexander Heinecke, Pradeep Dubey, Jesús Corbal, Nikita Shustrov, Roman Dubtsov, Evarist Fomenko, and Vadim O. Pirogov. 2018. Mixed Precision Training of Convolutional Neural Networks using Integer Operations. In *6th International Conference on Learning Representations, ICLR 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. https://openreview.net/forum?id=H135uzZ0-

[12] Christopher De Sa, Megan Leszczynski, Jian Zhang, Alana Marzoev, Christopher R Aberger, Kunle Olukotun, and Christopher Ré. 2018. High-accuracy low-precision training. *arXiv preprint arXiv:1803.03383* (2018).

[13] Alberto Delmas Lascorz, Patrick Judd, Dylan Malone Stuart, Zissis Poulos, Mostafa Mahmoud, Sayeh Sharify, Milos Nikolic, Kevin Siu, and Andreas Moshovos. 2019. Bit-Tactical: A Software/Hardware Approach to Exploiting Value and Bit Sparsity in Neural Networks. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '19)*. ACM, New York, NY, USA, 749–763. https://doi.org/10.1145/3297858.3304041

[14] Mario Drumond, Tao Lin, Martin Jaggi, and Babak Falsafi. 2018. Training DNNs with Hybrid Block Floating Point. In *Proceedings of the 32Nd International Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates Inc., USA, 451–461. http://dl.acm.org/citation.cfm?id=3326943.3326985

[15] Charles Eckert, Xiaowei Wang, Jingcheng Wang, Arun Subramaniyan, Ravi R. Iyer, Dennis Sylvester, David T. Blaauw, and Reetuparna Das. 2018. Neural Cache: Bit-Serial In-Cache Acceleration of Deep Neural Networks. In *45th ACM/IEEE Annual Intl' Symp. on Computer Architecture, ISCA 2018, Los Angeles, CA, USA, June 1-6, 2018*. 383–396. https://doi.org/10.1109/ISCA.2018.00040

[16] Desmond Elliott, Stella Frank, Khalil Sima'an, and Lucia Specia. 2016. Multi30K: Multilingual English-German Image Descriptions. arXiv:cs.CL/1605.00459

[17] Sameh Galal and Mark Horowitz. 2011. Energy-Efficient Floating-Point Unit Design. *IEEE Trans. Computers* 60, 7 (2011), 913–922. https://doi.org/10.1109/TC.2010.121

[18] David Goldberg. 1991. What every computer scientist should know about floating-point arithmetic. *Comput. Surveys* 23, 1 (1991), 5–48. https://doi.org/10.1145/103162.103163

[19] Ashish Gondimalla, Noah Chesnut, Mithuna Thottethodi, and T. N. Vijaykumar. 2019. SparTen: A Sparse Tensor Accelerator for Convolutional Neural Networks. In *Proceedings of the 52Nd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO '52)*. ACM, New York, NY, USA, 151–165. https://doi.org/10.1145/3352460.3358291

[20] Suyog Gupta, Ankur Agrawal, Kailash Gopalakrishnan, and Pritish Narayanan. 2015. Deep learning with limited numerical precision. In *International Conference on Machine Learning*. 1737–1746.

[21] Udit Gupta, Brandon Reagen, Lillian Pentecost, Marco Donato, Thierry Tambe, Alexander M. Rush, Gu-Yeon Wei, and David Brooks. 2019. MASR: A Modular Accelerator for Sparse RNNs. In *28th International Conference on Parallel Architectures and Compilation Techniques, PACT 2019, Seattle, WA, USA, September 23-26, 2019*. IEEE, 1–14. https://doi.org/10.1109/PACT.2019.00009

[22] Song Han, Xingyu Liu, Huizi Mao, Jing Pu, Ardavan Pedram, Mark A. Horowitz, and William J. Dally. 2016. EIE: Efficient Inference Engine on Compressed Deep Neural Network. In *Intl' Symp. on Computer Architecture*.

[23] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Trans. Interact. Intell. Syst.* 5, 4, Article Article 19 (Dec. 2015), 19 pages. https://doi.org/10.1145/2827872

[24] Greg Henry, Ping Tak Peter Tang, and Alexander Heinecke. 2019. Leveraging the bfloat16 Artificial Intelligence Datatype For Higher-Precision Computations. *2019 IEEE 26th Symposium on Computer Arithmetic (ARITH)* (Jun 2019). https://doi.org/10.1109/arith.2019.00019

[25] HewlettPackard. [n. d.]. CACTI. https://github.com/HewlettPackard/cacti.

[26] E. Hokenek, R. K. Montoye, and P. W. Cook. 1990. Second-generation RISC floating point with multiply-add fused. *IEEE Journal of Solid-State Circuits* 25, 5 (1990), 1207–1213. https://doi.org/10.1109/4.62143

[27] Roxana Istrate, Adelmo Cristiano Innocenza Malossi, Costas Bekas, and Dimitrios S. Nikolopoulos. 2018. Incremental Training of Deep Convolutional Neural Networks. *CoRR* abs/1803.10232 (2018). arXiv:1803.10232 http://arxiv.org/abs/1803.10232

[28] Patrick Judd, Jorge Albericio, Tayler Hetherington, Tor Aamodt, and Andreas Moshovos. 2016. Stripes: Bit-serial Deep Neural Network Computing . In *Intl' Symp. on Microarchitecture*.

[29] Dongyoung Kim, Junwhan Ahn, and Sungjoo Yoo. 2018. ZeNA: Zero-Aware Neural Network Accelerator. *IEEE Design Test* 35 (2018), 39–46.

[30] Urs Köster, Tristan J. Webb, Xin Wang, Marcel Nassar, Arjun K. Bansal, William H. Constable, Oğuz H. Elibol, Scott Gray, Stewart Hall, Luke Hornof, Amir Khosrowshahi, Carey Kloss, Ruby J. Pai, and Naveen Rao. 2017. Flexpoint: An Adaptive Numerical Format for Efficient Training of Deep Neural Networks. In *Proceedings of the 31st International Conference on Neural Information Processing Systems (NIPS'17)*. Curran Associates Inc., USA, 1740–1750. http://dl.acm.org/citation.cfm?id=3294771.3294937

[31] H. T. Kung, Bradley McDanel, and Sai Qian Zhang. 2019. Packing Sparse Convolutional Neural Networks for Efficient Systolic Array Implementations: Column Combining Under Joint Optimization. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, ASPLOS 2019, Providence, RI, USA, April 13-17, 2019*, Iris Bahar, Maurice Herlihy, Emmett Witchel, and Alvin R. Lebeck (Eds.). ACM, 821–834. https://doi.org/10.1145/3297858.3304028

[32] Pran Kurup and Taher Abbasi. 2011. *Logic Synthesis Using Synopsys* (2nd ed.). Springer Publishing Company, Incorporated.

[33] Alberto Delmas Lascorz, Sayeh Sharify, Isak Edo Vivancos, Dylan Malone Stuart, Omar Mohamed Awad, Patrick Judd, Mostafa Mahmoud, Milos Nikolic, Kevin Siu, Zissis Poulos, and Andreas Moshovos. 2019. ShapeShifter: Enabling Fine-Grain Data Width Adaptation in Deep Learning. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture, MICRO 2019, Columbus, OH, USA, October 12-16, 2019*. ACM, 28–41. https://doi.org/10.1145/3352460.3358295

[34] Jinmook Lee, Changhyeon Kim, Sanghoon Kang, Dongjoo Shin, Sangyeob Kim, and Hoi-Jun Yoo. 2019. UNPU: An Energy-Efficient Deep Neural Network Accelerator With Fully Variable Weight Bit Precision. *J. Solid-State Circuits* 54, 1 (2019), 173–185. https://doi.org/10.1109/JSSC.2018.2865489

[35] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. 2014. Microsoft COCO: Common Objects in Context. arXiv:cs.CV/1405.0312

[36] Shaoli Liu, Zidong Du, Jinhua Tao, Dong Han, Tao Luo, Yuan Xie, Yunji Chen, and Tianshi Chen. 2016. Cambricon: An Instruction Set Architecture for Neural Networks. In *2016 IEEE/ACM Intl' Conf. on Computer Architecture (ISCA)*.

[37] Paulius Micikevicius, Sharan Narang, Jonah Alben, Gregory F. Diamos, Erich Elsen, David García, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, and Hao Wu. 2018. Mixed Precision Training. In *6th International Conference on Learning Representations, 2018, Vancouver, BC, Canada, April 30 - May 3, 2018, Conference Track Proceedings*. https://openreview.net/forum?id=r1gs9JgRZ

[38] Inc. Micron Technology. [n. d.]. DDR4 Power Calculator 4.0. https://www.micron.com/~/media/documents/products/power-calculator/ddr4_power_calc.xlsm.

[39] Hesham Mostafa and Xin Wang. 2019. Parameter efficient training of deep convolutional neural networks by dynamic sparse reparameterization. In *International Conference on Machine Learning*. 4646–4655.

[40] NVIDIA. [n. d.]. Training With Mixed Precision. https://docs.nvidia.com/deeplearning/sdk/mixed-precision-training/index.html.

[41] Angshuman Parashar, Minsoo Rhu, Anurag Mukkara, Antonio Puglielli, Rangharajan Venkatesan, Brucek Khailany, Joel Emer, Stephen W. Keckler, and William J. Dally. 2017. SCNN: An Accelerator for Compressed-sparse Convolutional Neural Networks. In *Intl' Symp. on Computer Architecture (ISCA '17)*.

[42] Gennady Pekhimenko, Vivek Seshadri, Onur Mutlu, Phillip B. Gibbons, Michael A. Kozuch, and Todd C. Mowry. 2012. Base-Delta-Immediate Compression: Practical Data Compression for on-Chip Caches. In *Proceedings of the 21st International Conference on Parallel Architectures and Compilation Techniques (PACT '12)*. Association for Computing Machinery, New York, NY, USA, 377–388.

https://doi.org/10.1145/2370816.2370870

[43] Intel AI PlaidML. 2017. PlaidML. https://vertexai-plaidml.readthedocs-hosted.com/en/latest/index.html

[44] Eric Qin, Ananda Samajdar, Hyoukjun Kwon, Vineet Nadella, Sudarshan Srinivasan, Dipankar The, Bharat Kaul, and Tushar Krishna. 2020. SIGMA: A Sparse and Irregular GEMM Accelerator with Flexible Interconnects for DNN Training. In *IEEE International Symposium on High Performance Computer Architecture, HPCA 2020, San Diego, CA, USA, February 22-26, 2020*. IEEE, 58–70. https://doi.org/10.1109/HPCA47549.2020.00015

[45] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, Alexander C. Berg, and Li Fei-Fei. 2014. ImageNet Large Scale Visual Recognition Challenge. *CoRR* abs/1409.0575 (Sept. 2014).

[46] Charbel Sakr, Naigang Wang, Chia-Yu Chen, Jungwook Choi, Ankur Agrawal, Naresh Shanbhag, and Kailash Gopalakrishnan. 2019. Accumulation Bit-Width Scaling For Ultra-Low Precision Training Of Deep Networks. arXiv:cs.LG/1901.06588

[47] Charbel Sakr, Naigang Wang, Chia-Yu Chen, Jungwook Choi, Ankur Agrawal, Naresh Shanbhag, and Kailash Gopalakrishnan. 2019. Accumulation Bit-Width Scaling For Ultra-Low Precision Training Of Deep Networks. arXiv:cs.LG/1901.06588

[48] Sayeh Sharify, Alberto Delmas Lascorz, Mostafa Mahmoud, Milos Nikolic, Kevin Siu, Dylan Malone Stuart, Zissis Poulos, and Andreas Moshovos. 2019. Laconic deep learning inference acceleration. In *Proceedings of the 46th International Symposium on Computer Architecture, ISCA 2019, Phoenix, AZ, USA, June 22-26, 2019*. 304–317. https://doi.org/10.1145/3307650.3322255

[49] Sayeh Sharify, Alberto Delmas Lascorz, Kevin Siu, Patrick Judd, and Andreas Moshovos. 2018. Loom: Exploiting weight and activation precisions to accelerate convolutional neural networks. In *Proceedings of the 55th Annual Design Automation Conference*. ACM, 20.

[50] Hardik Sharma, Jongse Park, Naveen Suda, Liangzhen Lai, Benson Chau, Vikas Chandra, and Hadi Esmaeilzadeh. 2018. Bit Fusion: Bit-Level Dynamically Composable Architecture for Accelerating Deep Neural Network. In *ISCA*. IEEE Computer Society, 764–775.

[51] Jitesh R. Shinde and Suresh S. Salankar. 2015. VLSI implementation of bit serial architecture based multiplier in floating point arithmetic. In *2015 International Conference on Advances in Computing, Communications and Informatics, ICACCI 2015, Kochi, India, August 10-13, 2015*. IEEE, 1672–1677. https://doi.org/10.1109/ICACCI.2015.7275854

[52] Swagath Venkataramani, Ashish Ranjan, Subarno Banerjee, Dipankar Das, Sasikanth Avancha, Ashok Jagannathan, Ajaya Durg, Dheemanth Nagaraj, Bharat Kaul, Pradeep Dubey, and Anand Raghunathan. 2017. ScaleDeep: A Scalable Compute Architecture for Learning and Evaluating Deep Networks. In *Proceedings of the 44th Annual International Symposium on Computer Architecture (ISCA '17)*. ACM, New York, NY, USA, 13–26. https://doi.org/10.1145/3079856.3080244

[53] Naigang Wang, Jungwook Choi, Daniel Brand, Chia-Yu Chen, and Kailash Gopalakrishnan. 2018. Training Deep Neural Networks with 8-bit Floating Point Numbers. In *Proceedings of the 32Nd International Conference on Neural Information Processing Systems (NIPS'18)*. Curran Associates Inc., USA, 7686–7695. http://dl.acm.org/citation.cfm?id=3327757.3327866

[54] Shibo Wang and Pankaj Kanwar. 2019. BFloat16: The secret to high performance on Cloud TPUs. https://cloud.google.com/blog/products/ai-machine-learning/bfloat16-the-secret-to-high-performance-on-cloud-tpus

[55] Zelun Wang and Jyh-Charn Liu. 2019. Translating Math Formula Images to LaTeX Sequences Using Deep Neural Networks with Sequence-level Training. arXiv:cs.LG/1908.11415

[56] Tianjun Xiao, Jiaxing Zhang, Kuiyuan Yang, Yuxin Peng, and Zheng Zhang. 2014. Error-Driven Incremental Learning in Deep Convolutional Neural Network for Large-Scale Image Classification. In *Proceedings of the ACM International Conference on Multimedia, MM '14, Orlando, FL, USA, November 03 - 07, 2014*. 177–186. https://doi.org/10.1145/2647868.2654926

[57] Zhilin Yang, Zihang Dai, Yiming Yang, Jaime Carbonell, Ruslan Salakhutdinov, and Quoc V. Le. 2019. XLNet: Generalized Autoregressive Pretraining for Language Understanding. arXiv:cs.CL/1906.08237

[58] Rowan Zellers, Ari Holtzman, Hannah Rashkin, Yonatan Bisk, Ali Farhadi, Franziska Roesner, and Yejin Choi. 2019. Defending Against Neural Fake News. In *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett (Eds.). Curran Associates, Inc., 9054–9065. http://papers.nips.cc/paper/9106-defending-against-neural-fake-news.pdf

[59] Shijin Zhang, Zidong Du, Lei Zhang, Huiying Lan, Shaoli Liu, Ling Li, Qi Guo, Tianshi Chen, and Yunji Chen. 2016. Cambricon-X: An Accelerator for Sparse Neural Networks. In *Intl' Symp. on Microarchitecture*.

[60] Xuda Zhou, Zidong Du, Qi Guo, Chengsi Liu, Chao Wang, Xuehai Zhou, Ling Li, Tianshi Chen, and Yunji Chen. 2018. Cambricon-S: Addressing Irregularity in Sparse Neural Networks through a Cooperative Software/Hardware Approach. In *Intl' Symp. on Microarchitecture*.